Embedded Data Structure
(Lecture Note 9)

Applications of Linked List
- Namecard Management -

2025. 3. 31

Prof. Wonjun Kim

School of Electrical and Electronics Engineering

Review of Last Class

- **The heart of data structure and algorithm : linked list**
  - **It makes possible to access (manage) the memory "dynamically"**

    ! Remember "node" structure

    ! Recursive definition (typedef struct node { struct node *ptr;})

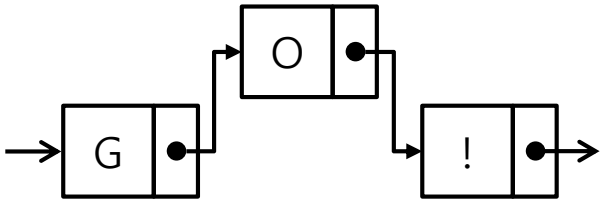
    ! **Draw the procedure** to easily understand linked lists

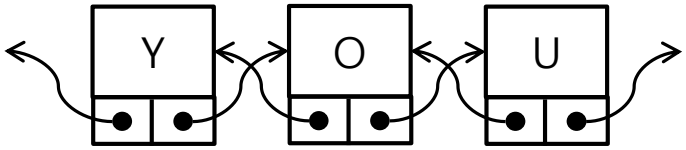
  - **Remember two types : simply linked list / doubly linked list**

    ! The main difference between two types is … (cons. and pros.)

    ! Insert : make link between new and previous nodes

    ! Delete : make free assigned memory





Name Card Management (1/11)

- **Simple management program**
  - **We use the simple linked list to implement it !**

    ➔ Basic elements : name, corporation, and telephone number

How to manage it efficiently ?

Our program may perform with …

1. Input name card
2. Delete name card
3. Search name card
4. Load name card : name_card.dat
5. Save name card : name_card.dat
6. Show the list of name card
7. End program

Image from https://www.facemediagroup.co.uk/business-print/business-card-printing

Name Card Management (2/11)

- **Result of our implementation will be …**

C:₩WINDOWS₩system32₩cmd.exe

```
Name Card Manager
-----------------------
1.Input name card
2.Delete name card
3.Search name card
4.Load name card
5.Save name card
6.Show list
7.Program end…
: selection operation->
```

[Menu]

C:₩WINDOWS₩system32₩cmd.exe

```
Name Card Manager
-----------------------
1.Input name card
2.Delete name card
3.Search name card
4.Load name card
5.Save name card
6.Show list
7.Program end…
: selection operation->1
Input namecard menu :
Input name : Wonjun Kim

Input corporation : Konkuk University

Input telephone number : 1234
```

Input inform.

1) Name
2) Affiliation
3) Tel. number

```
: selection operation->6

Name                 Corportation                    Telephone number
-------------------- ------------------------------- --------------------
Bill Evans           Label                                   5678
Wonjun Kim           Konkuk University                       1234
Name Card Manager
```

[Show the namecard list]

You can put more names !

Name Card Management (3/11)

- **Start with the definition of structure**
  - **For your understanding, we firstly consider the case of N = 3:**

```
#define NAME_SIZE   21
#define CORP_SIZE   31
#define TEL_SIZE    16
#define REC_SIZE
(NAME_SIZE + CORP_SIZE +
TEL_SIZE)
```

You do not need to consider the size of link (pointer)

```
typedef struct _card{
    char name[NAME_SIZE];
    char corp[CORP_SIZE];
    char tel[TEL_SIZE];
    struct _card *next;
}card;

card *head, *tail;

void init_card()
{
    head = (card *)calloc(1, sizeof(card));
    tail = (card *)calloc(1, sizeof(card));

}
```

Name Card Management (4/11)

KU KONKUK UNIVERSITY

▪ Main function implementation

- **Input cards : you can input arbitrary name cards**

```
void input_card()
{
    card *t;
    t = (card *)calloc(1, sizeof(card));

    printf("₩Input namecard menu : ");
    printf("₩nInput name : ");
    gets_s(t->name);
    printf("₩nInput corporation : ");
    gets_s(t->corp);
    printf("₩nInput telephone number : ");
    gets_s(t->tel);

    t->next = head->next;
    head->next = t;
}
```

Get the input string

What does it mean ?

Name Card Management (5/11)

- **Main function implementation – cont'd**
  - **Delete and Search functions**

```
int delete_card(char *s)
{
    card *t, *p;
    p = head;
    t = p->next;
    while(strcmp(s, t->name) != 0 &&
            t != tail){
        p = p->next;
        [          ]
    }
    if(t == tail)
        return 0;
    p->next = t->next;
    free(t);
    return 1;
}
```

```
card *search_card(char *s)
{
    card *t;
    t = head->next;
    while(strcmp(s, t->name) != 0 &&
            t != tail){
        [          ]
    }
    if(t == tail)
        return NULL;
    else
        return [   ];
}
```

➡ **Fill the blanks (just review !)**

KU KONKUK UNIVERSITY

▪ Main function implementation – cont'd

- **Save function : using "fwrite()"**

*** fread() and fwrite()**

```
Form : size_t fread(void *ptr,
                    size_t size,
                    size_t nitems,
                    FILE *stream);

Form : size_t fwrite(void *ptr,
                     size_t size,
                     size_t nitems,
                     FILE *stream);
```

Size of data

Number of data

```
void save_card(char *s)
{
    FILE *fp;
    card *t;
    fopen_s(&fp, s, "wb");

    t = head->next;
    while(t != tail)
    {
        fwrite(t, REC_SIZE, 1, fp);
        t = t->next;
    }

    fclose(fp);
}
```

KU KONKUK UNIVERSITY

Main function implementation – cont'd

- **Load function : using "fread()"**

```
void load_card(char *s)
{
    FILE *fp;
    card *t, *u;
    fopen_s(&fp, s, "rb");

    t = head->next;
    while(t != tail)
    {
        u = t;
        t = t->next;
        free(u);
    }

    head->next = tail;
    while(1)
    {
        t = (card *)calloc(1, sizeof(card));
        if(!fread(t, REC_SIZE, 1, fp))
        {
            free(t);
            break;
        }
        t->next = head->next;
        head->next = t;
    }

    fclose(fp);
}
```

What does it mean ?

Name Card Management (8/11)

KU KONKUK UNIVERSITY

▪ Difference between text and binary files

- **For example, we want to print "123456" using text and binary**

1) fprintf : '1', '2', '3', '4', '5', '6' are stored as string

※ We can read as integers using fscanf function (conversion required)

2) fwrite : '1', '2', '3', '4', '5', '6' are stored as binary

※ Data is saved as binary format (we cannot read it !)

*** What are cons and pros between text and binary files ?**

Name Card Management (9/11)

- **Print the result**
  - **Using the print_header() and print_card()**

```
void print_header(FILE *f)
{
    fprintf(f, "₩nName                    "
               "Corportation                    "
               "Telephone number");
    fprintf(f, "₩n---------------------     "
               "-------------------------------     "
               "---------------------");
}
```

```
void print_card(card *t, FILE *f)
{
    fprintf(f, "₩n%-20s     %-30s      %-15s", t->name, t->corp, t->tel);
}
```

Name Card Management (10/11)

KU KONKUK UNIVERSITY

▪ Make a menu function

- **Make a simple function to select operations you want !**

```
int select_menu()
{
    int j;
    char s[10];
    printf("₩nName Card Manager");
    printf("₩n-----------------------");
    printf("₩n1. Input name card");
    printf("₩n2. Delete name card");
    printf("₩n3. Search name card");
    printf("₩n4. Load name card");
    printf("₩n5. Save name card");
    printf("₩n6. Show list");
    printf("₩n7. Program end...");
    do{
        printf("₩n: selection operation ->");
        j = atoi(gets_s(s));
    }while(j < 0 || j > 7);

    return j;
}
```

➡ **Why using do-while ?**

Name Card Management (11/11)

- **Total implementation**

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
...
void main()
{
    char *fname = "NameCard.dat";
    char name[NAME_SIZE];
    int i;
    card *t;

    init_card();

    while((i=select_menu()) != 7) {
```

[redacted]

```
    }
    printf("₩n₩nProgram ends...₩n");
}
```

```
switch(i){
    case 1:
        input_card();
        break;
    case 2:
        printf("₩nInput name to delete -> ");
        gets_s(name);
        if(!delet_card(name)) printf("₩nCan't fine the name...");
        break;
    case 3:
        printf("₩nInput name to search -> ");
        gets_s(name);
        t = search_card(name);
        if(t == NULL) break;
        print_header(stdout);
        print_card(t, stdout);
        break;
    case 4:
        load_cards(fname);
        break;
    case 5:
        save_cards(fname);
        break;
    case 6:
        t = head->next;
        print_header(stdout);
        while(t != tail) {print_card(t, stdout); t = t->next}
        break;
}
```

Summary

- **Simple linked list is very useful for data management**
  - **Remember the node and link structure**
  - **Namecard management : try to understand all the procedure**

    (Of course, you need to complete your implementation !)